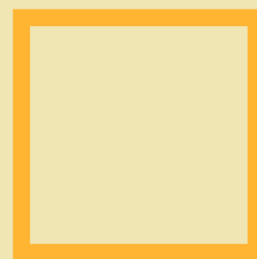
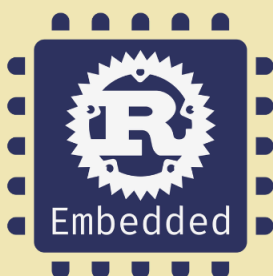




The Embedded
Rustacean
Blog

+ Follow



Sharing Data Among Tasks in Rust Embassy: Synchronization Primitives



Omar Hiari

Jan 10, 2023 • 📖 11 min read



Table of contents

Introduction

The List of Primitives

The Use Cases

Reading/Writing from Multiple Tasks

- › The AtomicU32 Type
- › The Blocking Mutex Type

Reading/Writing Across async Tasks

- › The async Mutex Type

Wait for Value Change

- › The Signal Type
- › The Channel Type
- › The PubSubChannel Type
- › The Pipe Type

Conclusion

Show less ^

Introduction

If you haven't experienced it yet, dealing with global variables in embedded Rust could be a rough experience. Though for good reason I would say. The challenge stems from making sure that variables are shared in a "safe" manner among threads to prevent cases like data races. However, using embassy, the experience is rectified through several synchronization primitives provided through the embassy-sync crate. Though the question is, why are there so many different primitives? and which should you use? The answer lies in how you plan to share the data. Meaning, do you want to only share it among tasks in a blocking manner? or do you want to share it among async tasks? better yet, do you want the primitives to handle value changes?



Please note that the full code in this post is available on the [apollo-labsdev Nucleo-F401RE](#) git repo. Additionally, the hardware used in this post is the [Nucleo-F401RE board](#).

Stay in the know and skyrocket your embedded Rust learning curve by subscribing to our newsletter below. In subscribing, you'll get an instant 10% off Simplified Embedded Rust.

The Embedded Rustacean Newsletter

The source for everything embedded Rust. Curated insights for embedded developers. 2x a month.

Subscribe Now



The List of Primitives

The embassy-sync crate provides the following primitives (along with the associated description from the [documentation](#)):

- `Channel` - A Multiple Producer Multiple Consumer (MPMC) channel. Each message is only received by a single consumer.
- `PubSubChannel` - A broadcast channel (publish-subscribe) channel. Each message is received by all consumers.
- `Signal` - Signalling latest value to a single consumer.
- `Mutex` - Mutex for synchronizing state between asynchronous tasks.
- `Pipe` - Byte stream implementing `embedded_io` traits.

There are also `waker` primitives which are listed below. I won't be covering these here, but you should be aware of them as it is efficient to know that `waker` primitives exist for the future.



- `WakerRegistration` - Utility to register and wake a `Waker`.
- `AtomicWaker` - A variant of `WakerRegistration` accessible using a non-mut API.
- `MultiWakerRegistration` - Utility registering and waking multiple `Waker`'s.

So this all looks great, but how or when would I choose which to use?

The Use Cases

The use of the different primitives can be split into three cases:

1. **Reading/Writing from/to multiple tasks:** This is a common case where all one would need is a simple read/write from/to a variable in multiple tasks.
2. **Reading/Writing across `async` tasks:** These are cases where the value that is being held by a variable needs to be held across `async` tasks. Consequently, this means holding a lock while `await` ing.
3. **Wait for Value Change:** These are cases where more than read/write is needed. In particular, a receiving task waiting for a change in value.

In the following sections the constructs available under each category are covered.

Reading/Writing from Multiple Tasks

The `AtomicU32` Type

You might have noticed that `AtomicU32` is not in the list presented earlier. That is because `AtomicU32` is not really an `embassy-sync` primitive but rather a `core::sync` primitive. I decided to add it to this post for the sake of completion as I found it really helpful myself. The `AtomicU32` is relatively easy to use and is good enough if you just want to share a simple value among tasks. However, `AtomicU32` works only for types that are `u32` or less in size. If the size is larger, one needs to defer to using a global blocking `Mutex` which is presented after.

Below is an example of how the `AtomicU32` type `SHARED` variable and initialize it to



0. Following that, `store()` and `load()` methods are used to store and load values to/from the global context.

In the code below, in the `async_task` task, the value in `SHARED` is incremented every 1 second. The `load()` method is used to retrieve the current value and the `store()` method is used to update the shared value in the global context.

As shown as well, the `store()` and `load()` methods require an `Ordering` enum argument to be specified. The `Ordering` enum refers to the way atomic operations synchronize memory which I chose `Relaxed`. For more detail, one can refer to the full list of options [here](#). There is even a more detailed explanation of ordering [here](#).

COPY

```
static SHARED: AtomicU32 = AtomicU32::new(0);

#[embassy_executor::task]
async fn async_task() {
    loop {
        // Load value from global context, modify and store
        let shared_var = SHARED.load(Ordering::Relaxed);
        SHARED.store(shared_var.wrapping_add(1), Ordering::Relaxed);
        Timer::after(Duration::from_millis(1000)).await;
    }
}

#[embassy_executor::main]
async fn main(spawner: Spawner) {
    // Initialize and create handle for device peripherals
    let p = embassy_stm32::init(Default::default());
    //Configure UART
    let mut usart = UartTx::new(p.USART2, p.PA2, NoDma, Config::default());
    // Create empty String for message
    let mut msg: String<8> = String::new();
    // Spawn async blinking task
    spawner.spawn(async_task()).unwrap();

    loop {
        // Load value from global context, modify and store
        let s
        // Wait 1 second
        Timer::after(Duration::from_millis(1000)).await;
```

```

        // Format value for printing
        core::writeln!(&mut msg, "{:02}", shared).unwrap();
        // Transmit Message
        usart.blocking_write(msg.as_bytes()).unwrap();
        msg.clear();
    }
}

```

The Blocking Mutex Type

The use case of blocking Mutex is similar to AtomicU32 . The difference is that the blocking Mutex supports values larger than u32 . Keep in mind that the blocking Mutex will block threads waiting for the lock to become available. Additionally, the blocking Mutex does not hold the lock between await points.

The code below repeats the earlier example of AtomicU32 only using the blocking Mutex instead. In the below code note the following differences:

- When instantiating the shared global variable, a RawMutex type needs to be defined. Here I chose ThreadModeMutex . What to chose depends on the context in which you're using the mutex. In our context which we choose does not matter much. More detail can be found [here](#).
- In the global variable installation, the u32 type inside the Mutex is wrapped with a RefCell . This is to allow interior mutability of the value that is wrapped in the Mutex .
- To access SHARED the lock method is used to obtain a lock to the global shared value. lock provides access to the locked variable in a closure where the value is mutated.

COPY

```

use core::cell::RefCell;
use embassy_sync::blocking_mutex::Mutex;

static SHARED: Mutex<ThreadModeRawMutex, RefCell<u32>> = Mutex::new(Re

#[embassy_executor::task]
async fn async_task() {
    loop {

```



```

        // Load value from global context, modify and store
        SHARED.lock(|f| {
            let val = f.borrow_mut().wrapping_add(1);
            f.replace(val);
        });
        Timer::after(Duration::from_millis(1000)).await;
    }
}

#[embassy_executor::main]
async fn main(spawner: Spawner) {
    // Initialize and create handle for device peripherals
    let p = embassy_stm32::init(Default::default());
    //Configure UART
    let mut usart = UartTx::new(p.USART2, p.PA2, NoDma, Config::default());
    // Create empty String for message
    let mut msg: String<8> = String::new();
    // Spawn async blinking task
    spawner.spawn(async_task()).unwrap();

    loop {
        // Wait 1 second
        Timer::after(Duration::from_millis(1000)).await;
        // Obtain updated value from global context
        let shared = SHARED.lock(|f| f.clone().into_inner());
        core::writeln!(&mut msg, "{:02}", shared).unwrap();
        // Transmit Message
        usart.blocking_write(msg.as_bytes()).unwrap();
        msg.clear();
    }
}

```

Reading/Writing Across async Tasks

The async Mutex Type

So how does a blocking `Mutex` differ from an `async Mutex`? A blocking `Mutex` lock is not held across `await` points. With an `async Mutex`, one can `lock()` and `wait` accordingly.

The code below achieves the same as the earlier code with a slight modification. First, note that the `RefCell` is no longer required and that the shared variable is not accessed through a closure. Instead, a lock methods `await` s until the lock is acquired and doesn't let it go until the scope ends. If you notice the scope of the lock in the `async_task` there is a `Timer::after(Duration::from_millis(1000)).await;` line that is inserted. It would be interesting to play around with the delay value in this line and see how the code behaves. The point is to demonstrate that as you increase the value, the lock will remain held by `async_task` preventing `SHARED` in the main task from being accessed.

COPY

```
use embassy_sync::blocking_mutex::raw::ThreadModeRawMutex;
use embassy_sync::mutex::Mutex;

static SHARED: Mutex<ThreadModeRawMutex, u32> = Mutex::new(0);

#[embassy_executor::task]
async fn async_task() {
    loop {
        {
            let mut shared = SHARED.lock().await;
            *shared = shared.wrapping_add(1);
            Timer::after(Duration::from_millis(1000)).await;
        }
        Timer::after(Duration::from_millis(1000)).await;
    }
}

#[embassy_executor::main]
async fn main(spawner: Spawner) {
    // Initialize and create handle for device peripherals
    let p = embassy_stm32::init(Default::default());
    //Configure UART
    let mut usart = UartTx::new(p.USART2, p.PA2, NoDma, Config::default);
    // Create empty String for message
    let mut msg: String<8> = String::new();
    // Spawn async blinking task
    spawner.spawn(async_task);

    loop {
```




```

        // Wait 1 second
        Timer::after(Duration::from_millis(1000)).await;
        // Obtain updated value from global context
        let shared = SHARED.lock().await;
        core::writeln!(&mut msg, "{:02}", *shared).unwrap();
        // Transmit Message
        usart.blocking_write(msg.as_bytes()).unwrap();
        msg.clear();
    }
}

```

Wait for Value Change

The Signal Type

`Signal` provides for a simple case where **one** value needs to be buffered/sent to another task. This is done by sending a "Signal" that a new value is available.

The code below shows a usage example where a value of 5 is sent every second from the `async_task` to the main task. The `signal` method updates the value and sets up a signal. The `wait` method in the main task `await`s until a signal is received and updates `val` and prints it to the console. In this case, the new value signaled is always the same which is 5. Note here a difference with the `AtomicU32` is that before I didn't check or wait for the value to change/update. I only grabbed whatever was stored in the shared global variable.

COPY

```

use embassy_sync::signal::Signal;
static SHARED: Signal<ThreadModeRawMutex, u32> = Signal::new();

#[embassy_executor::task]
async fn async_task() {
    loop {
        SHARED.signal(5);
        Timer::after(Duration::from_millis(1000)).await;
    }
}

```



```
#[embassy_executor::main]
async fn main(spawner: Spawner) {
    // Initialize and create handle for device peripherals
    let p = embassy_stm32::init(Default::default());
    //Configure UART
    let mut usart = UartTx::new(p.USART2, p.PA2, NoDma, Config::default);
    // Create empty String for message
    let mut msg: String<16> = String::new();
    // Spawn async blinking task
    spawner.spawn(async_task()).unwrap();

    loop {
        let val = SHARED.wait().await;
        core::writeln!(&mut msg, "Signal Detected {:02}", val).unwrap();
        //Transmit Message
        usart.blocking_write(msg.as_bytes()).unwrap();
        msg.clear();
    }
}
```

The Channel Type

`Channel` expands on `Signal` in which it allows for multiple values to be buffered in a queue. As such, when instantiating a `Channel` the queue size needs to be defined. A `Channel` allows for multiple producers to write to a queue and multiple consumers to read. However, there can be only one reader for a value. This means that it's a first come first serve type of construct that once the first consumer reads a value, it's not available for other consumers anymore.

In the example below, a value of 2 is chosen for the `Channel` size. This is because I am going to allow two tasks to buffer/publish values to the `Channel` and one task to retrieve values. In the code example below note how the tasks `async_task_one` and `async_task_two` use the `send` method to send the values of 1 and 2 to the `SHARED` `Channel`. The `main` task, on the other, hand uses the `recv` method to retrieve the values buffered in `SHARED` and prints them to the serial monitor. If you run this code, you'll notice that the value of 1 will be printed twice to the serial monitor followed by a single instance of 2. This is because the time delay in `async_task_one` is a second.

```

use embassy_sync::channel::Channel;

//Declare a channel of 2 u32s
static SHARED: Channel<ThreadModeRawMutex, u32, 2> = Channel::new();

#[embassy_executor::task]
async fn async_task_one() {
    loop {
        SHARED.send(1).await;
        Timer::after(Duration::from_millis(500)).await;
    }
}

#[embassy_executor::task]
async fn async_task_two() {
    loop {
        SHARED.send(2).await;
        Timer::after(Duration::from_millis(1000)).await;
    }
}

#[embassy_executor::main]
async fn main(spawner: Spawner) {
    // Initialize and create handle for device peripherals
    let p = embassy_stm32::init(Default::default());
    //Configure UART
    let mut usart = UartTx::new(p.USART2, p.PA2, NoDma, Config::default);
    // Create empty String for message
    let mut msg: String<16> = String::new();
    // Spawn async blinking task
    spawner.spawn(async_task_one()).unwrap();
    spawner.spawn(async_task_two()).unwrap();

    loop {
        let val = SHARED.recv().await;
        core::writeln!(&mut msg, "{:02}", val).unwrap();
        //Transmit Message
        usart.blocking_write(msg.as_bytes()).unwrap();
        msg.clear();
    }
}

```



The PubSubChannel Type

The `PubSubChannel` is an expansion of the `Channel` type in which multiple consumers can now access the value. In the `Channel` type, if one consumer read a value, it's no longer available for others. With the `PubSubChannel` this is no longer an issue. However, what actually could happen in a `PubSubChannel` is that a certain consumer/subscriber can miss out on a value in the queue because a new value was pushed in. If that happens, the `PubSubChannel` provides an error signaling that occurrence.

The example below repeats the `Channel` example above, though using a `PubSubChannel` instead. Note how `pub1` and `pub2` are declared as publishers to `SHARED` and use the `publish_immediate` method to send/publish values to the global queue. Correspondingly, `sub` is declared as a subscriber to `SHARED` and uses the `next_message_pure` method to access the values in the global queue.

COPY

```
use embassy_sync::pubsub::PubSubChannel;

//Declare a pubsub channel with a capacity of 2 and 1 subscriber and 2 |
static SHARED: PubSubChannel<ThreadModeRawMutex, u32, 2, 2, 2> = PubSub

#[embassy_executor::task]
async fn async_task_one() {
    let pub1 = SHARED.publisher().unwrap();
    loop {
        pub1.publish_immediate(1);
        Timer::after(Duration::from_millis(500)).await;
    }
}

#[embassy_executor::task]
async fn async_task_two() {
    let pub2 = SHARED.publisher().unwrap();
    loop {
        pub2.publish_immediate(2);
        Timer::after(Duration::from_millis(1000)).await;
    }
}
```



```
#[embassy_executor::main]
async fn main(spawner: Spawner) {
    // Initialize and create handle for device peripherals
    let p = embassy_stm32::init(Default::default());
    //Configure UART
    let mut usart = UartTx::new(p.USART2, p.PA2, NoDma, Config::default()
    // Create empty String for message
    let mut msg: String<16> = String::new();
    // Spawn async blinking task
    spawner.spawn(async_task_one()).unwrap();
    spawner.spawn(async_task_two()).unwrap();

    let mut sub = SHARED.subscriber().unwrap();

    loop {
        let val = sub.next_message_pure().await;
        core::writeln!(&mut msg, "{:02}", val).unwrap();
        //Transmit Message
        usart.blocking_write(msg.as_bytes()).unwrap();
        msg.clear();
    }
}
```

The Pipe Type

A Pipe is more or less just like a Channel though restricted to buffering of u8 types.

Conclusion

Dealing with global variables shared among threads in embedded Rust (or Rust in general) can be a hassle normally. In embassy, however different types are provided through the embassy-sync crate to facilitate value sharing among threads. This post furnishes the different types available through embassy-sync and how they can be used. Have any questions/comments? Share your thoughts in the comments below 📌.



Remember, if you want access to more Embedded Rust material, subscribe to our new newsletter:

The Embedded Rustacean Newsletter

The source for everything embedded Rust. Curated insights for embedded developers. 2x a month.

Subscribe Now



Did you find this article valuable?

Support **The Embedded Rustacean Blog** by becoming a sponsor. Any amount is appreciated!



Sponsor

[Learn more about Hashnode Sponsors](#)

Rust

Internet of Things

Beginner Developers

embedded

ARTICLE SERIES

Embassy



Embedded Rust & Embassy: GPIO Button Controlled Blinking

This blog post is the first one of a multi-part series of posts where I will explore various periphe...

2

Embedded Rust & Embassy: UART Serial Communication

This blog post is the second of a multi-part series where I will explore various peripherals of the ...

... [Show all 4 posts](#)



7

Embedded Rust and Embassy: DMA Controllers

This blog post is the seventh of a multi-part series of posts where I explore various peripherals in...

8

Sharing Data Among Tasks in Rust Embassy: Synchronization Primitives

Introduction If you haven't experienced it yet, dealing with global variables in embedded Rust could...



©2025 The Embedded Rustacean Blog

[Archive](#) • [Privacy policy](#) • [Terms](#)



Powered by Hashnode - Build your developer hub.

[Start your blog](#)

[Create docs](#)

